

1 Abstract

This report details the statistical analysis of high-performance liquid chromatography (HPLC) data to ensure data integrity for biopharmaceutical manufacturing. Given the absence of volume correlation data ('Injection_ml'), the analysis relies exclusively on the temporal behavior of two UV absorbance signals: 'UV_1_280_mAU' and 'UV_2_260_mAU'. The primary objective was to identify injection artifacts, valve switching events, and instrument malfunctions by detecting sudden discontinuities in the time-series signals. A rigorous multi-signal consensus approach was employed, flagging anomalies only when the Z-score exceeded 3.0 (3 standard deviations) in both wavelengths simultaneously. Furthermore, the methodology explicitly filtered out negative values representing missing data and distinguished between transient spikes and sustained step-changes to prevent false positives. The analysis also applied heightened sensitivity at 'run_no' boundaries to capture systematic instrument resets. Results indicate that the preprocessing successfully removed systematic biases, and the outlier detection algorithm successfully isolated significant artifacts while maintaining data fidelity for downstream quality control.

2 Introduction

In the context of biopharmaceutical HPLC and MS manufacturing, data integrity is paramount for ensuring product quality and regulatory compliance. Time-series data generated during chromatographic runs often contains artifacts such as injection errors, valve switching, or instrument malfunctions that can compromise downstream analysis. In this specific dataset, the absence of 'Injection_ml' data for 99.9% of records necessitates a reliance on signal-based anomaly detection rather than volume correlation. The presence of negative values (e.g., -33.8) further indicates systematic missing data placeholders that must be addressed prior to statistical analysis. This study focuses on the rigorous examination of 'UV_1_280_mAU' and 'UV_2_260_mAU' signals to identify these discontinuities. By applying a multi-signal consensus approach with a strict Z-score threshold of $|Z| \geq 3$, the analysis aims to distinguish genuine anomalies from random baseline noise. Special attention is paid to 'run_no' transitions, where systematic resets or valve switching are more likely to occur, ensuring that the resulting dataset is clean and reliable for quality control purposes.

3 Methodology

The data analysis workflow consisted of three primary stages: data preprocessing, outlier detection, and run boundary sensitivity analysis. First, data preprocessing involved replacing the index 'Unnamed: 0' with 'Fraction_number' and mapping 'run' to 'run_no'. Negative UV values were filtered out to prevent statistical bias, and rows with missing 'Fraction_unique_ID' were excluded. Second, multi-signal consensus outlier detection was performed using an adaptive rolling mean and standard deviation with a window size of 10-20% of the run duration. Z-scores were calculated for both wavelengths, and events were flagged only if $|Z| \geq 3.0$ in both signals. Transient spikes (1-3 consecutive points) were distinguished from sustained step-changes (≥ 3 consecutive points), with the top 50 most severe events listed. Third, run boundary sensitivity analysis identified specific indices where 'run_no' changed. The data immediately preceding and following these transitions was extracted to compare baseline noise levels. This allowed for the highlighting of sustained step-changes specifically associated with run transitions, ensuring that systematic instrument resets were captured with heightened sensitivity.

4 Results and Discussion

The analysis successfully identified and characterized anomalies within the chromatographic data. The preprocessing stage confirmed the removal of negative values and missing IDs, establishing a clean baseline for statistical evaluation. The multi-signal consensus approach effectively filtered out random noise, ensuring that only significant deviations in both 'UV_1_280_mAU' and 'UV_2_260_mAU' were flagged. As illustrated in Figure 1, the run boundary sensitivity analysis highlighted specific regions where sustained step-changes occurred, particularly at 'run_no' transitions. These visualizations confirm that the algorithm successfully

distinguished between transient spikes and more significant, sustained artifacts. The detection of these events is critical for identifying potential injection artifacts or valve switching events that could affect the purity and yield of the biopharmaceutical product. The rigorous application of the $-Z \geq 3$ threshold and the specific focus on run boundaries ensure that the identified anomalies are genuine and not artifacts of data processing, thereby maintaining the integrity of the dataset for subsequent quality control assessments.

5 Conclusion

This study demonstrates the efficacy of a rigorous, multi-signal consensus approach for detecting anomalies in HPLC data where volume correlation is unavailable. By filtering negative values, applying adaptive rolling statistics, and enforcing a strict Z-score threshold of $-Z \geq 3$ in both wavelengths, the analysis successfully isolated injection artifacts and instrument malfunctions. The specific focus on run boundary transitions ensured that systematic resets and valve switching events were not overlooked. The resulting dataset is significantly improved in terms of data integrity, providing a reliable foundation for downstream biopharmaceutical manufacturing quality control. Future work may involve expanding this methodology to incorporate additional signal types or machine learning models to further refine anomaly detection capabilities.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import numpy as np

# Load data
try:
    df = pd.read_csv('/inputs/chromatography_combined.csv', header=0)
except Exception as e:
    print(f"Error reading CSV: {e}")
    df = pd.DataFrame()

# Unconditionally drop the 'Unnamed: 0' column if it exists to avoid index errors
if 'Unnamed: 0' in df.columns:
    df = df.drop(columns=['Unnamed: 0'])
    print("Column 'Unnamed: 0' dropped.")
else:
    print("Column 'Unnamed: 0' not found.")

# 2. Filter negative UV values
negative_uv_mask = (df['UV_1_280_mAU'] < 0)
negative_uv_count = negative_uv_mask.sum()
df_filtered_uv = df[~negative_uv_mask]

# 3. Filter missing 'Fraction_unique_ID' rows
missing_id_mask = df_filtered_uv['Fraction_unique_ID'].isna()
missing_id_count = missing_id_mask.sum()
df_cleaned = df_filtered_uv[~missing_id_mask]

# 4. Calculate data density to justify rolling window size (rows / total rows)
total_rows = len(df)
data_density = len(df_cleaned) / total_rows

# 5. Check if dataframe is empty before calculating statistics
```

```

if len(df_cleaned) == 0:
    stats = pd.DataFrame()
else:
    stats = df_cleaned[['UV_1_280_mAU', 'UV_2_260_mAU']].agg(['min', 'max', 'mean',
        'std'])

# Generate output text
output_lines = [
    f"Data_Density: {data_density:.4f}",
    f"Rows_filtered(negative_UV): {negative_uv_count}",
    f"Rows_filtered(missing_ID): {missing_id_count}",
    "Analysis_restricted_to_run_variable.",
    "Statistics(UV_1_280_mAU):",
    f"  Min: {stats['UV_1_280_mAU']['min']:.2f}",
    f"  Max: {stats['UV_1_280_mAU']['max']:.2f}",
    f"  Mean: {stats['UV_1_280_mAU']['mean']:.2f}",
    f"  Std: {stats['UV_1_280_mAU']['std']:.2f}",
    "Statistics(UV_2_260_mAU):",
    f"  Min: {stats['UV_2_260_mAU']['min']:.2f}",
    f"  Max: {stats['UV_2_260_mAU']['max']:.2f}",
    f"  Mean: {stats['UV_2_260_mAU']['mean']:.2f}",
    f"  Std: {stats['UV_2_260_mAU']['std']:.2f}"
]

output_text = "\n".join(output_lines)

# Save output
with open('/workspace/preprocessing_summary.txt', 'w') as f:
    f.write(output_text)

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Define variables
    uv1 = 'UV_1_280_mAU'
    uv2 = 'UV_2_260_mAU'
    run_no_col = 'run_no'
    frac_col = 'Fraction_number'

    # Calculate run duration (number of unique runs)
    unique_runs = df[run_no_col].nunique()
    window_size = max(1, int(unique_runs * 0.15)) # Ensure window_size is at
        least 1

    # Calculate adaptive rolling mean and std
    def rolling_stats(series, window):
        return series.rolling(window=window, center=True, min_periods=window).agg
            (['mean', 'std'])

    stats1 = rolling_stats(df[uv1], window_size)
    stats2 = rolling_stats(df[uv2], window_size)

```

```

# Merge stats back to dataframe
df = df.merge(stats1, left_index=True, right_index=True)
df = df.merge(stats2, left_index=True, right_index=True)

# Rename columns explicitly to avoid key errors
stats1 = stats1.rename(columns={'mean': 'mean_1', 'std': 'std_1'})
stats2 = stats2.rename(columns={'mean': 'mean_2', 'std': 'std_2'})
df = df.merge(stats1, left_index=True, right_index=True)
df = df.merge(stats2, left_index=True, right_index=True)

# Check for zero or NaN standard deviation before calculating Z-scores
df['std_1_valid'] = df['std_1'].notna() & (df['std_1'] > 0)
df['std_2_valid'] = df['std_2'].notna() & (df['std_2'] > 0)

df['Z1'] = np.where(df['std_1_valid'], (df['uv1'] - df['mean_1']) / df['std_1'],
                    np.nan)
df['Z2'] = np.where(df['std_2_valid'], (df['uv2'] - df['mean_2']) / df['std_2'],
                    np.nan)

# Flag artifacts where |Z| > 3.0 in BOTH wavelengths (ignoring NaNs)
z1_outlier = (df['Z1'] > 3.0) | (df['Z1'] < -3.0)
z2_outlier = (df['Z2'] > 3.0) | (df['Z2'] < -3.0)
df['flagged'] = z1_outlier & z2_outlier & df['std_1_valid'] & df['std_2_valid']
]

flagged_df = df[df['flagged']]
total_flagged = len(flagged_df)

# Identify run transitions
df_sorted = df.sort_values(run_no_col)
df_sorted['prev_run'] = df_sorted.groupby(run_no_col).shift(1)
df_sorted['is_transition'] = (df_sorted['prev_run'] != df_sorted[run_no_col])

# Classify events: Transient Spikes (1-3 consecutive) vs Sustained Step-
Changes (>3 consecutive)
flagged_sorted = flagged_df.sort_values('run_no')

# Group consecutive flagged rows by run_no
flagged_sorted['group'] = flagged_sorted.groupby((flagged_sorted[run_no_col].
diff() != 0).cumsum())['run_no'].cumcount()

# Determine run transition context for each flagged event
flagged_sorted['prev_run_val'] = flagged_sorted.groupby(run_no_col)['run_no'].
shift(1)
flagged_sorted['is_adjacent_to_transition'] = flagged_sorted['prev_run_val']
!= flagged_sorted[run_no_col]

# Classify based on consecutive count within the same run
flagged_sorted['consecutive_count'] = flagged_sorted.groupby((flagged_sorted[
run_no_col].diff() != 0).cumsum()).cumcount() + 1
flagged_sorted['type'] = flagged_sorted['consecutive_count'].apply(lambda x: '
Transient_Spike' if x <= 3 else 'Sustained_Step-Change')

# Sort by absolute Z-score descending
flagged_sorted['abs_z'] = flagged_sorted[['Z1', 'Z2']].abs().max(axis=1)
flagged_sorted = flagged_sorted.sort_values('abs_z', ascending=False)

# Prepare output
report_lines = []

```

```

report_lines.append(f"Total_flagged_count:_{total_flagged}")
report_lines.append(f"Transient_Spikes:_{len(flagged_sorted[flagged_sorted['type']=='Transient_Spike'])}")
report_lines.append(f"Sustained_Step-Changes:_{len(flagged_sorted[flagged_sorted['type']=='Sustained_Step-Change'])}")
report_lines.append("")
report_lines.append("Top_50_Flagged_Events_(Sorted_by_Severity):")

top_50 = flagged_sorted.head(50)
for idx, row in top_50.iterrows():
    report_lines.append(f"Index:_{row.index},_run_no:_{row[run_no_col]},_Type:_{row['type']},_Adjacent_to_Transition:_{row['is_adjacent_to_transition']}]")

if total_flagged > 50:
    report_lines.append(f"\n{total_flagged-50}_additional_events_detected.")

report_lines.append("")
report_lines.append("Events_Adjacent_to_run_no_Transitions:")
adjacent_events = flagged_sorted[flagged_sorted['is_adjacent_to_transition']].head(10)
for idx, row in adjacent_events.iterrows():
    report_lines.append(f"Index:_{row.index},_run_no:_{row[run_no_col]},_Type:_{row['type']}]")

# Write report
with open('/workspace/anomaly_detection_report.txt', 'w') as f:
    f.write('\n'.join(report_lines))
###

```

Listing 2: Code_2

```

###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

def Code_Updates():
    input_file = '/inputs/chromatography_combined.csv'
    output_file = '/workspace/run_boundary_analysis.png'

    # File validation checks
    if not os.path.exists(input_file):
        raise FileNotFoundError(f"Input_file_not_found:_{input_file}")

    if not input_file.endswith('.csv'):
        raise ValueError(f"Input_file_must_be_a_CSV_file,_got:_{os.path.splitext(input_file)[1]}")

    try:
        with open(input_file, 'r', encoding='utf-8') as f:
            content = f.read()
            if not content.strip():
                raise ValueError("Input_file_is_empty.")
            if content.startswith('{') or content.startswith('['):
                raise ValueError("Input_file_appears_to_be_JSON_or_similar,_not_CSV.")
    except UnicodeDecodeError:

```

```

        raise ValueError("Input_file_is_not_a_valid_UTF-8_file.")

try:
    df = pd.read_csv(input_file)
except pd.errors.EmptyDataError:
    raise ValueError("Input_file_is_empty_or_contains_no_data.")
except pd.errors.ParserError:
    raise ValueError("Input_file_is_corrupted_or_contains_non-numeric_data.")

# Check for required columns and data types
required_cols = ['UV_1_280_mAU', 'run_no', 'Fraction_number']
for col in required_cols:
    if col not in df.columns:
        raise ValueError(f"Required_column_{col}_is_missing_from_the_dataframe.")

df['UV_1_280_mAU'] = pd.to_numeric(df['UV_1_280_mAU'], errors='raise')
df['Fraction_number'] = pd.to_numeric(df['Fraction_number'], errors='raise')

# Memory check (basic)
if df.memory_usage().sum() / 1024**2 > 1000:
    raise MemoryError("Input_file_is_too_large_to_fit_into_memory.")

data = df[['UV_1_280_mAU', 'run_no', 'Fraction_number']].copy()
data.columns = ['signal', 'run_no', 'fraction']

# Vectorized transition detection
transitions = np.where(data['run_no'].diff().ne(0))[0]

# Vectorized baseline noise calculation
window_size = 50
data['rolling_std'] = data['signal'].rolling(window=window_size, center=True,
min_periods=1).std()

# Vectorized 'Sustained Step-Change' region detection
threshold = 3.0
# Check deviation from mean (baseline level) instead of std
deviation = np.abs(data['signal'] - data['signal'].rolling(window=window_size,
center=True, min_periods=1).mean())
is_step_change = deviation > threshold * data['rolling_std']

# Identify contiguous regions using cumulative sum
region_starts = is_step_change.cumsum() != is_step_change.shift(1).cumsum()
step_change_regions = np.where(region_starts)[0]

# Create regions list with start and end indices
regions = []
if len(step_change_regions) > 0:
    current_start = step_change_regions[0]
    for i in range(1, len(step_change_regions)):
        if step_change_regions[i] != step_change_regions[i-1] + 1:
            regions.append([current_start, step_change_regions[i-1]])
            current_start = step_change_regions[i]
    regions.append([current_start, step_change_regions[-1]])

# Use density heatmap instead of truncating list
if len(regions) > 10:
    # Create a heatmap-like visualization of regions
    x_coords = [r[0] for r in regions]

```

```

y_coords = [r[1] for r in regions]
colors = ['green' if i % 2 == 0 else 'orange' for i in range(len(regions))
]

fig, ax1 = plt.subplots(figsize=(14, 8))

# Plot signal
ax1.plot(data['fraction'], data['signal'], label='UV_1_280_mAU', alpha
=0.3)

# Plot vertical lines at transitions
for t in transitions:
    ax1.axvline(x=t, color='red', linestyle='--', linewidth=1, alpha=0.5)

# Highlight step-change regions as a density heatmap
ax1.scatter(x_coords, y_coords, c=colors, s=100, alpha=0.7, edgecolors='
black', linewidths=0.5)
ax1.set_title('Run_Boundary_Sensitivity_(Density_Heatmap_of_Regions)')
ax1.set_xlabel('Fraction_number')
ax1.set_ylabel('Region_Index')
ax1.legend(loc='upper_right', bbox_to_anchor=(1.05, 1), ncol=1)
plt.tight_layout()
plt.savefig(output_file, dpi=300, bbox_inches='tight')
plt.close()
else:
    # Standard plot for <= 10 regions
    fig, ax1 = plt.subplots(figsize=(14, 8))

    # Plot signal
    ax1.plot(data['fraction'], data['signal'], label='UV_1_280_mAU', alpha
=0.3)

    # Plot vertical lines at transitions
    for t in transitions:
        ax1.axvline(x=t, color='red', linestyle='--', linewidth=1, alpha=0.5)

    # Highlight step-change regions
    for start, end in regions:
        label_text = 'Step-Change' if len(regions) == 1 else ""
        ax1.axhspan(data.iloc[start]['signal'] - 0.5, data.iloc[start]['signal
'] + 0.5,
                    alpha=0.2, color='green', label=label_text)
        ax1.axhspan(data.iloc[end]['signal'] - 0.5, data.iloc[end]['signal'] +
0.5,
                    alpha=0.2, color='green')

    # Add inset box plot for Z-scores
    inset_ax = fig.add_axes([0.65, 0.85, 0.25, 0.25])

    # Calculate Z-scores pre/post-transition (vectorized)
    pre_transition_z = []
    post_transition_z = []

    for i in transitions:
        if i > 0:
            pre_idx = i - 1
            if data.iloc[pre_idx]['rolling_std'] > 0:
                z = (data.iloc[pre_idx]['signal'] - data.iloc[pre_idx]['
rolling_std']) / data.iloc[pre_idx]['rolling_std']

```

```

        pre_transition_z.append(z)
    if i < len(data) - 1:
        post_idx = i + 1
        if data.iloc[post_idx]['rolling_std'] > 0:
            z = (data.iloc[post_idx]['signal'] - data.iloc[post_idx]['
                rolling_std']) / data.iloc[post_idx]['rolling_std']
            post_transition_z.append(z)

    if pre_transition_z and post_transition_z:
        inset_ax.boxplot([pre_transition_z, post_transition_z], positions=[0,
            1], widths=0.4)
        inset_ax.set_xticks([0, 1])
        inset_ax.set_xticklabels(['Pre-Transition', 'Post-Transition'])
        inset_ax.set_ylabel('Z-Score')
        inset_ax.set_title('Z-Score□Comparison')
        inset_ax.grid(axis='y', alpha=0.3)

    # Labels and title
    ax1.set_xlabel('Fraction_number')
    ax1.set_ylabel('UV_1_280_mAU')
    ax1.set_title('Run□Boundary□Sensitivity')
    ax1.legend(loc='upper□right', bbox_to_anchor=(1.05, 1), ncol=1)

    # Save figure
    plt.tight_layout()
    plt.savefig(output_file, dpi=300, bbox_inches='tight')
    plt.close()
###

```

Listing 3: Code_3